Developing and Debugging Source Generators

Christian Nagel

https://www.cninnovation.com

Christian Nagel

- 20+ years of experience with C# and .NET
- Author of 20+ books on C# and .NET (latest: "Modern C# in .NET 10")
- Microsoft MVP for C# and .NET
- Speaker at global conferences (including BASTA! for 20+ years)
- Passionate about language design and developer productivity

Agenda

1. **Roslyn & Compiler APIs** — How the C# compiler works
2. **Source Generator Architecture** — Types and pipeline
3. **Implementing a Generator** — Step-by-step
4. **Debugging & Testing** — Making generators maintainable
5. **Practical generators** — Generating data, accessing files, call-site rewriting, accessing private members

Best Practices & Gotchas

Why Source Generators?

Problems they solve:

```
// 😩 Before: Manual boilerplate (INotifyPropertyChanged)
public class Product : INotifyPropertyChanged
{
  private decimal _price;

  public event PropertyChangedEventHandler? PropertyChanged;

  public decimal Price
  {
    get => _price;
    set { if (_price != value) { _price = value; OnPropertyChanged(); } }
  }
  // ... repeat for every property!
}
```

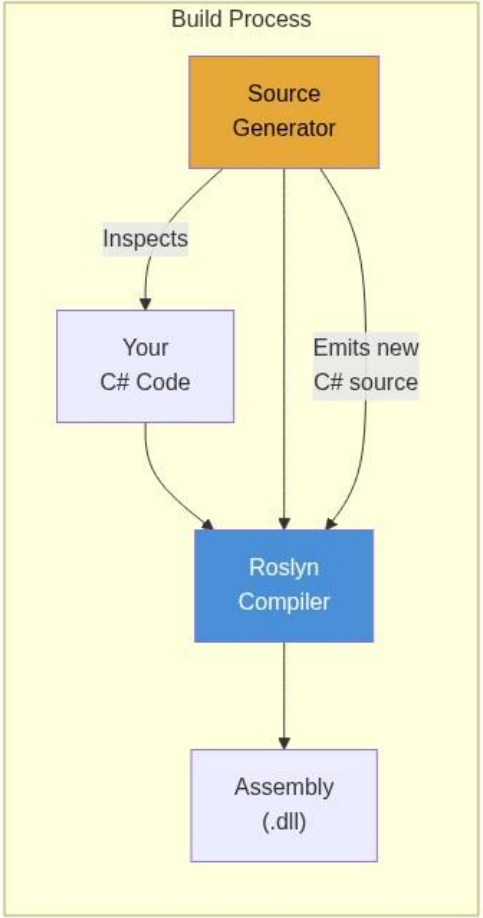
Solved

The source generator writes it for you!

```
// 😊 After
[NotifyPropertyChanged]
public partial class Product
{
  [Observable] private string _name = "";
  [Observable] private decimal _price;
}
```

What Are Source Generators?

- Run **during compilation** — not at runtime
- Can inspect your code and **emit new C# source files**
- Generated code is part of your project — **debuggable and navigable**
- Two generations: **v1 (ISourceGenerator)** (obsolete) and **v2 (IIncrementalGenerator)**

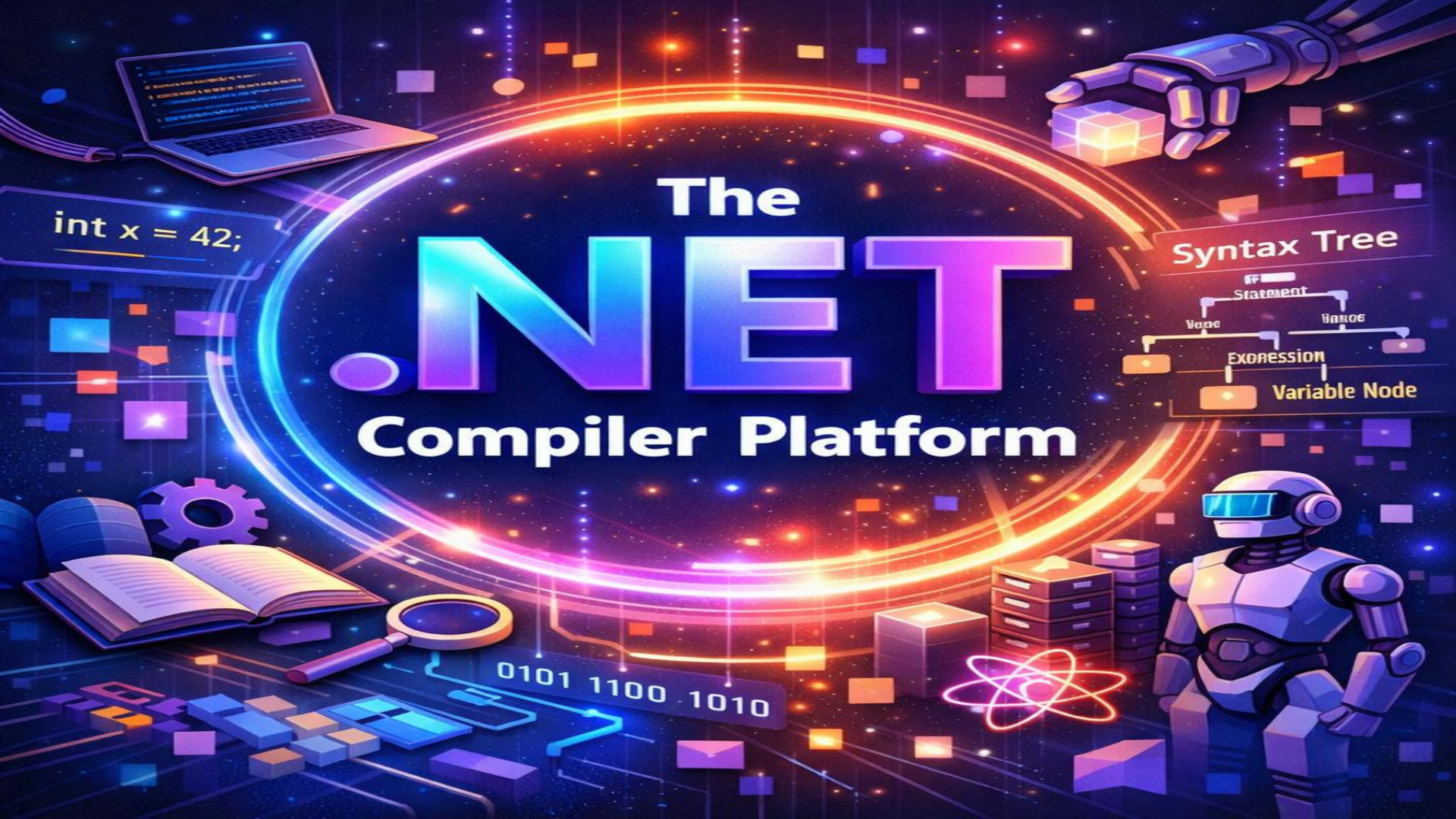


Design Goals

- Produce C# code that's added to the compilation
- Additive only - cannot modify existing user code
- May access additional files (also non-C# files)
- Run un-ordered - a source generator can't access output of other generators
- User specifies generators (NuGet packages)
- Generates create a pipeline of transformations (syntax → semantics → output)

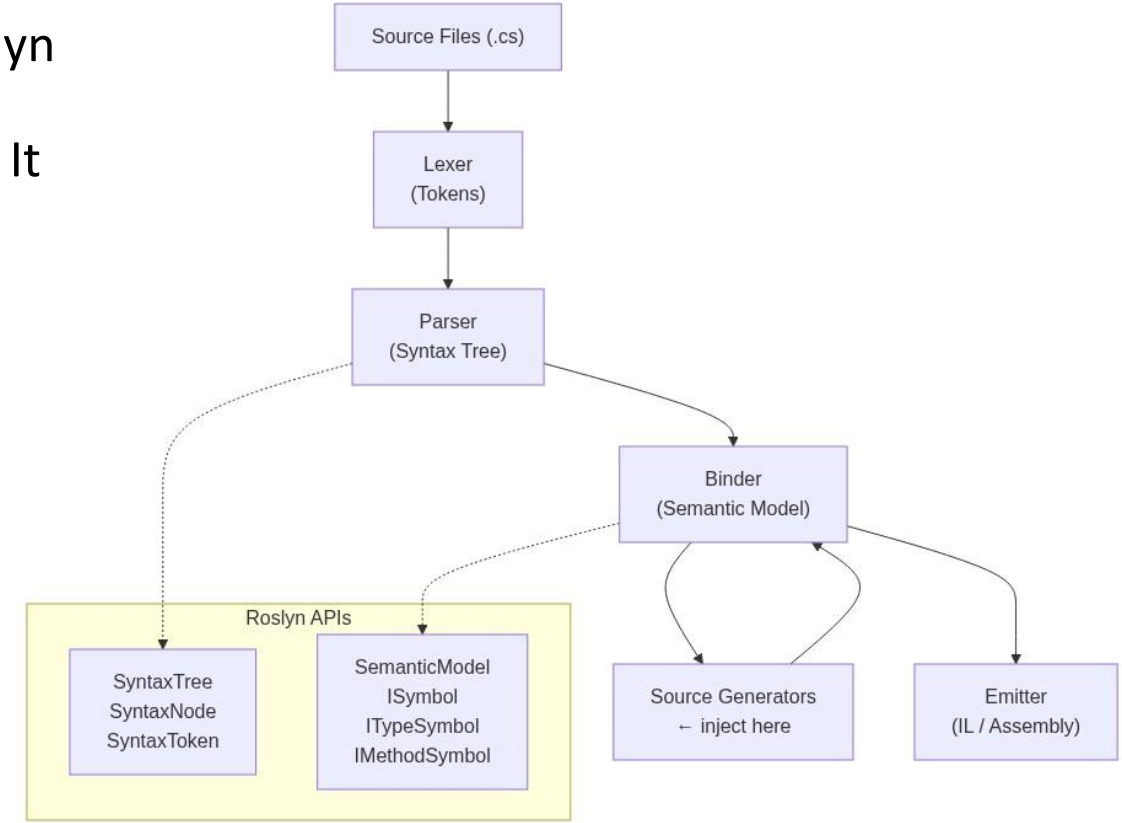
Non-design

- Language features (e.g. records, unions)
- Code-rewriting
  - Optimization
  - IL weaving
  - ~~Logging injection~~
  - ~~Call-site rewriting~~


The
.NET
Compiler Platform
int x = 42;
Syntax Tree
Statement
Expression
Variable Node
0101 1100 1010

Roslyn Compiler Architecture

Introduced with C# 6.0, Roslyn is the open-source compiler platform for C# and VB.NET. It provides rich APIs for code analysis and generation.



Roslyn Key Concepts

Syntax vs. Semantics

```
// Given this code:
var x = 42;
string s = x.ToString();

// SyntaxTree — what does the text look like?
// Sees: local variable declaration, assignment, method call
// Knows nothing about types

// SemanticModel — what does it mean?
// Knows: x is System.Int32
//        ToString() is Object.ToString()
//        s is System.String
```

Demo

Using the Roslyn APIs

- Syntax tree
- Semantic model

Roslyn Key Types (1)

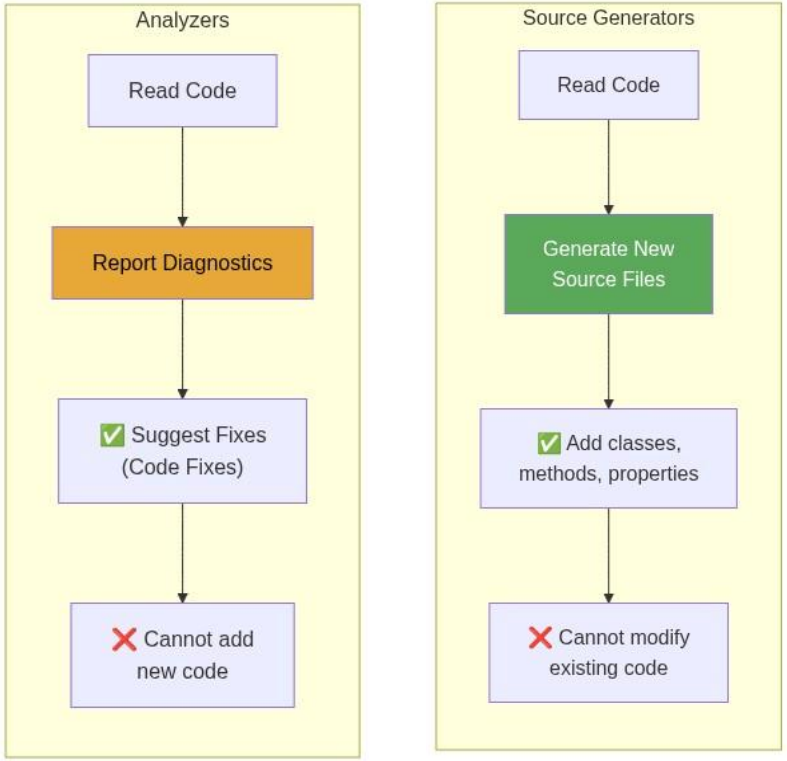
API	Purpose
SyntaxTree	AST representation of source
SyntaxNode	Any node (class, method, expression)
ISymbol	Semantic info (type, method, property)
ITypeSymbol	Type-specific semantic info
SemanticModel	Bridge between syntax and semantics

Roslyn Key Types (2)

API	Purpose
Compilation	All source files + references

Analyzers vs. Source Generators

Two complementary Roslyn tools with different roles



When to Use Each (1)

Use an Analyzer when:

- ✅ Enforcing coding standards (naming conventions, patterns)
- ✅ Detecting API misuse or bugs (null reference, disposal)
- ✅ Suggesting refactorings or simplifications
- ✅ Offering quick fixes for common issues

When to Use Each (2)

Use a Source Generator when:

- ✅ Repetitive boilerplate follows a template
- ✅ Code generation based on attributes or conventions
- ✅ Need AOT-compatible alternatives to reflection
- ✅ Creating companion types or extension methods

Use BOTH together

```
// Analyzer: "Add [Observable] attribute" code fix
// Generator: Actually generates the property from [Observable] field

[NotifyPropertyChanged]
public partial class Product
{
  [Observable] private string _name = "";  // ← Analyzer suggests adding this
  //                                           Generator creates public property
}
```

Create Your First

ROSLYN SOURCE GENERATOR

Your First Source Generator

Project structure:

dotnet new sourcegen -o HelloGenerator

```
HelloGen/                    ← Generator project
├── HelloGen.csproj          ← Special project settings!
└── HelloGenImpl.cs


MyApp/                          ← Consumer project
├── MyApp.csproj
└── Program.cs
```

Source Generator Project File

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <!-- Modern approach (VS 2022 17.7+ / VS 2026+): out-of-process generators -->
    <TargetFramework>net8.0</TargetFramework>  <!-- or net9.0, net10.0 ✅ -->
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <EnforceExtendedAnalyzerRules>true</EnforceExtendedAnalyzerRules>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.CodeAnalysis.CSharp"
                      Version="5.x.x" PrivateAssets="all" />
  </ItemGroup>
</Project>
```

Reference the Source Generator Project

In the consuming project, add a project reference:

```
<ItemGroup>
  <ProjectReference Include="../SourceGen.csproj"
      OutputItemType="Analyzer" ReferenceOutputAssembly="false" />
</ItemGroup>
```

Generator assembly is not copied to output — it's loaded by the compiler, not at runtime.

Create a Source Generator NuGet package

The binary output of the generator project must be included in the consuming project as an analyzer reference. When packing a NuGet package, include the generator assembly in the analyzers/dotnet/cs folder:

```
<ProperyGroup>
  <DevelopmentDependency>true</DevelopmentDependency>
</PropertyGroup>
<ItemGroup>
  <None Include="$(OutputPath)\$(AssemblyName).dll"
      Pack="true"
      PackagePath="analyzers/dotnet/cs" />
</ItemGroup>
```

1. Implement Interface

```
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp.Syntax;

[Generator]
public class HelloGenImpl : IIncrementalGenerator
{
  public void Initialize(IncrementalGeneratorInitializationContext context)
  {
```

2. Register Post-Initialization Output

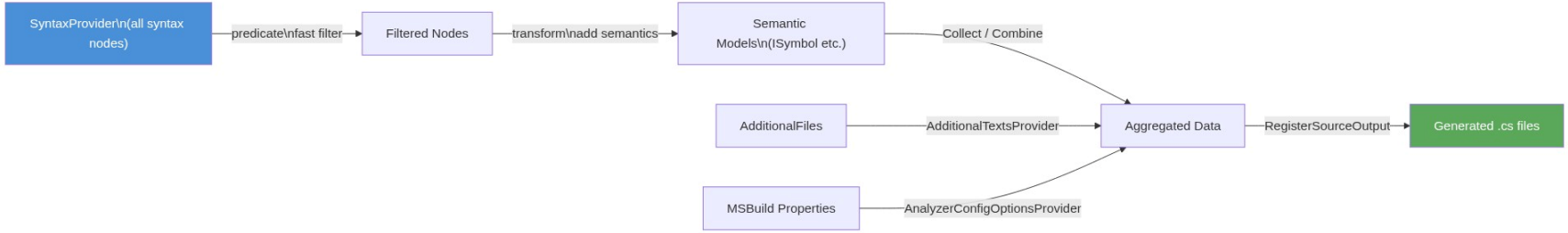
For code that doesn't depend on user code (e.g. helper attributes), you can register a "post-initialization output" that runs once before any user code is

```
context.RegisterPostInitializationOutput(ctx =>
{
  ctx.AddSource("GenerateInfoAttribute.g.cs", """
    namespace Generated
    {
      [System.AttributeUsage(System.AttributeTargets.Class)]
      public sealed class GenerateInfoAttribute : System.Attribute { }
    }
    """);
});
```

3. Build an incremental pipeline

```
IncrementalValuesProvider<MarkedClassInfo> classes =
    context.SyntaxProvider.CreateSyntaxProvider(
      predicate: static (node, _) => node is ClassDeclarationSyntax { AttributeLists.Count > 0
},
      transform: static (ctx, _) => (MarkedClassInfo)GetMarkedClassInfo(ctx))
      .Collect();

  // Step 3: Register source output
  context.RegisterSourceOutput(classes, GenerateCode);
}
```

Incremental Pipeline — Flow



Key principle: Each step is **cached**. Only re-run when inputs change.

Debugging Source Generators

- Visual Studio 2022 17.7+ and VS 2026+ run generators **out-of-process**
- Attach and debug is possible, but unit testing is usually more efficient for development

Unit tests - verify generated source code

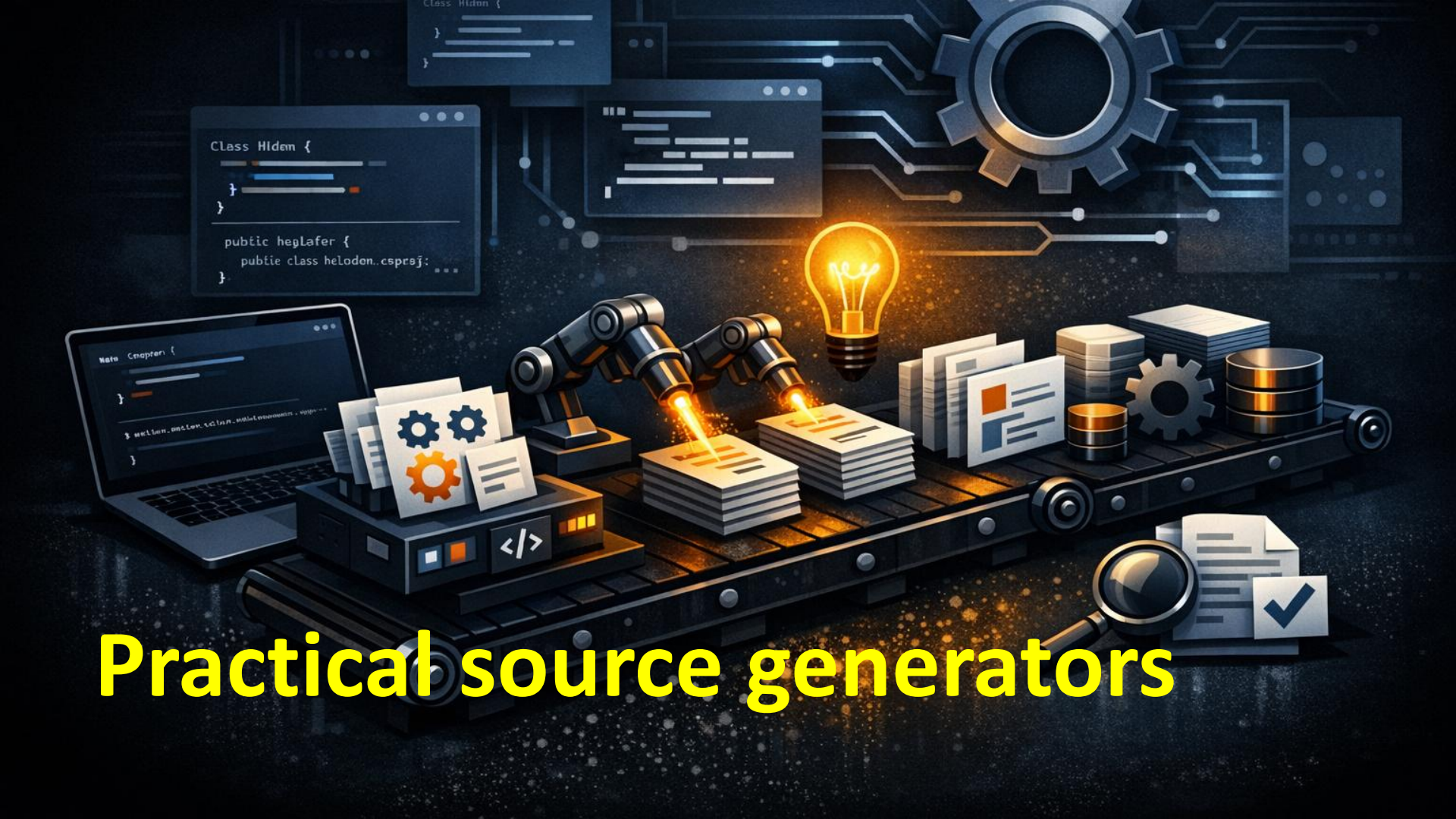
```
[Fact]
public void GeneratesExpectedCode()
{
  // Arrange: source code that uses the generator
  // Act: run the generator
  (var output, diag) = TestHelper.RunGenerator(source);

  // Assert: no errors, generated code contains expected results
}
```

Rosyln types for generators

- CSharpGeneratorDriver
  - Orchestrates source generator execution
- GeneratorDriver
  - Abstract base class for language-specific drivers
  - Managed pipeline state
  - RunGeneratorsAndUpdateCompilation() method

Snapshot Testing

- Great for checking and fixing results from source generation
- Captures generated code as "snapshots" in test files
- On test failure, shows diff between expected and actual generated code
- Fast feedback loop



Practical source generators

Sample data generator

Use case: generate factory classes for sample items

```
[DataSource(EntityName = "User", Count = 5)]
public class User
{
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public int Age { get; set; }
    public bool IsActive { get; set; }
    public DateTime CreatedAt { get; set; }
    public UserRole Role { get; set; }
}
```

Syntax Provider (1)

- CreateSyntaxProvider
  - Preferred way to hook into the pipeline
- Predicate
  - Filter and transform efficiently
  - Runs on every syntax node, so must be fast
- Transform
  - Runs only on nodes that pass predicate
  - Can access semantic model for deeper analysis

Syntax Provider (2)

- Where
  - Further filter transform results
- Collect
  - Aggregate the data into an ImmutableArray for output

Incremental Value Provider

- Declarative pipeline stage
- Computes a value once per compilation
- Lazy - only runs when needed
- Can be cached - if inputs don't change, output is reused
- Composable - Select/Combine/Where/...

Accessing External Files

Add external data sources to be used by the source generator

```
[DataSource(
  EntityName = "User",
  Count = 5,
  ConfigurationFile = "User.datasource.json")]
```

Improvement - Cache File Content

- Incremental value provider chain
- File change detection
- Per-file tracking
- Compares inputs between builds
- Caches outputs for unchanged inputs
- Re-executes transforms only with changed inputs

Improvement - Attribute Markers

- Typical use case
- Eliminates manual interaction with attributes
- Better caching performance

```
var classDeclarations = context.SyntaxProvider
  .ForAttributeWithMetadataName(
```

Special Features of Source Generators

C# Version Support

Generate Code for Multiple C# Versions
C#9, C10, C# 11 & C# 12)

```
public class AutoGen {
+
  void GenerateCode() {
  }:  |/Generated code
  |}
```

Accessing Private Members

Bypass Encapsulation & Access Private Fields / Methods

```
FieldInfo secretField = tupree(TargetClass). GetField"secretData";
BindingFlags.NonPublic);
secretField.SetValue(obj, 42);
```

Call-Site Rewriting with Interceptors

Modify Method Calls with Interceptors

Use the C# version of the caller

Use the caller's C# version for generated code to ensure compatibility:

```
var languageVersionProvider = context.ParseOptionsProvider
  .Select(static (parseOptions, ct)
    => GetLanguageVersionInfo(parseOptions));
```

Partial events for Weak events

```
// User code
WeakEvent(AutoCleanup = true, CleanupThreshold = 5)]
public partial event Action<string> WeakMessageReceived;
```

```
// Generated code
public partial event Action<string> WeakMessageReceived
{
  add
  {
  }
  remove
  {
  }
}
```

Initialize private members using UnsafeAccessor

C# UnsafeAccessor attribute allows source generators to access private members of the target class, enabling scenarios like deserializing, reading from database without reflection

```
// Generated code
[UnsafeAccessor(UnsafeAccessorKind.Field, Name = "_title")]
private static extern ref string GetBookTitleField(Book instance);

[UnsafeAccessor(UnsafeAccessorKind.Method, Name = "set_Publisher")]
private static extern void SetBookWithPrivateSettersPublisher(
  BookWithPrivateSetters instance, string value);
```

Call-site rewriting

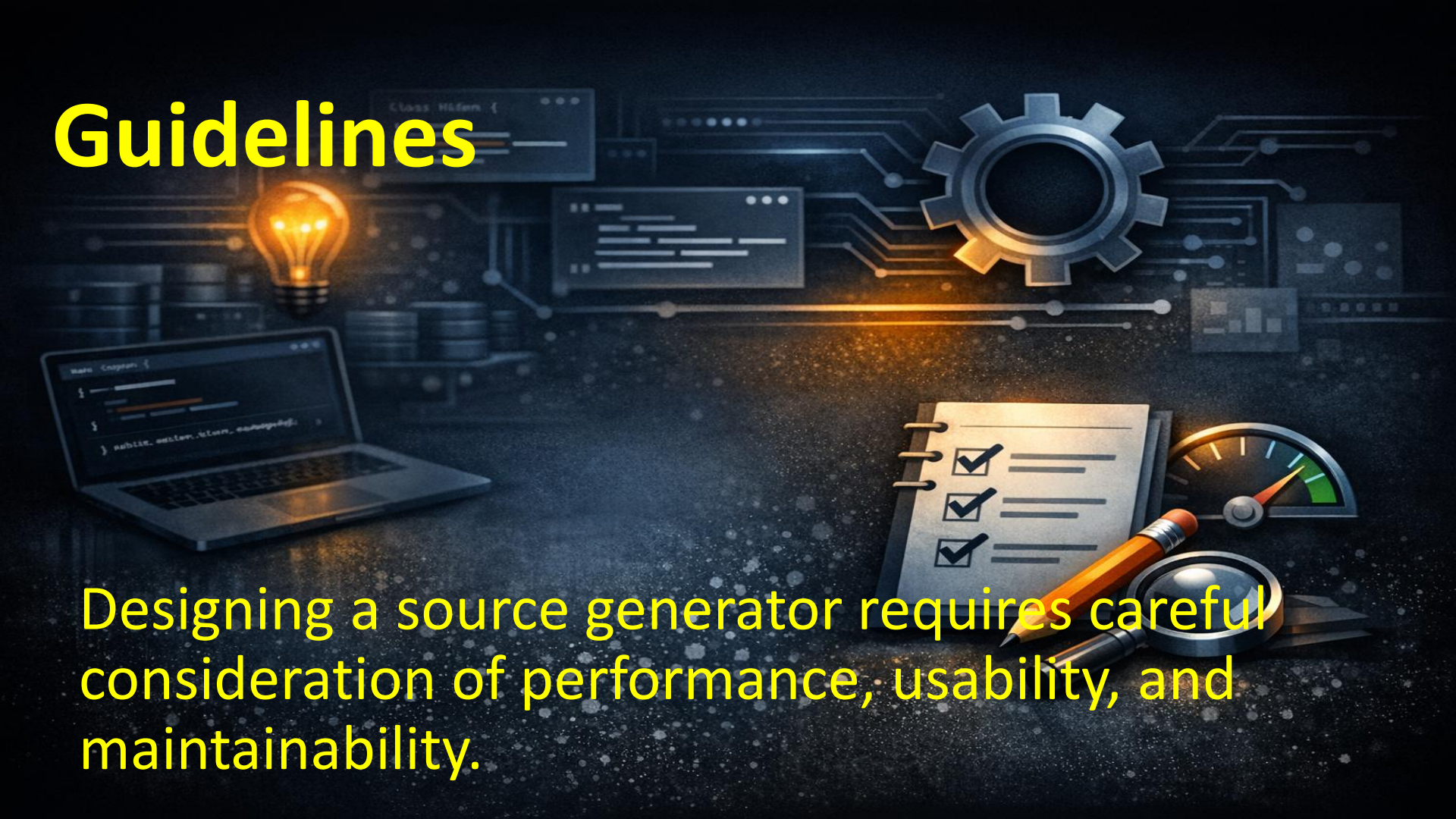
- InterceptableLocation attribute
- Roslyn API: SemanticModel.GetInterceptionLocation

```
global::System.Runtime.CompilerServices.InterceptsLocationAttribute(1,
"qPii2RCL4JQb4YSFXL7gkCUGAABQcm9ncmFtLmNz")]
    public static async System.Threading.Tasks.Task<int> ProcessAsync(string data)
```

.NET Built-in Source Generators

- System.Text.Json source generator
- Logging LoggerMessage source generator
- Regex compiled source generator
- COM Interop source generator
- gRPC
- Razor Views and Components
- ...

Guidelines



Designing a source generator requires careful consideration of performance, usability, and maintainability.

Mode	Trigger Frequency	Responsiveness	Performance Impact	Use Case
Automatic (Full)	After every keystroke	🟢 High - Immediate feedback	🔴 High CPU/memory usage	Small projects, actively developing generators
Balanced (Default)	Smart intervals (typing pauses, file save)	🟡 Medium - Slight delay	🟡 Moderate resource usage	Most projects, best for daily work
Conservative (Minimal)	Only on explicit build (F5/Ctrl+Shift+B)	🔴 Low - Manual trigger needed	🟢 Minimal - generators idle	Large projects, battery saving, Copilot

Source Generator Modes (2)

Configure in: Tools → Options → Text Editor → C# → Advanced → Source Generators

Performance Best Practices

Rules:

1. **predicate** — fast, syntax-only, no semantic model
2. **transform** — use EquatableArray<T> for cache hits
3. **Collect()** — aggregate before RegisterSourceOutput
4. Make data types **value-comparable** (implement IEquatable<T>)

```
// ✅ Predicate: syntax only (fast)
predicate: static (node, _) =>
  node is ClassDeclarationSyntax { AttributeLists.Count: > 0 }

// ❌ Avoid in predicate: semantic model (slow)
predicate: static (node, ctx) =>
  ctx.SemanticModel.GetDeclaredSymbol(node) is INamedTypeSymbol // SLOW!
```

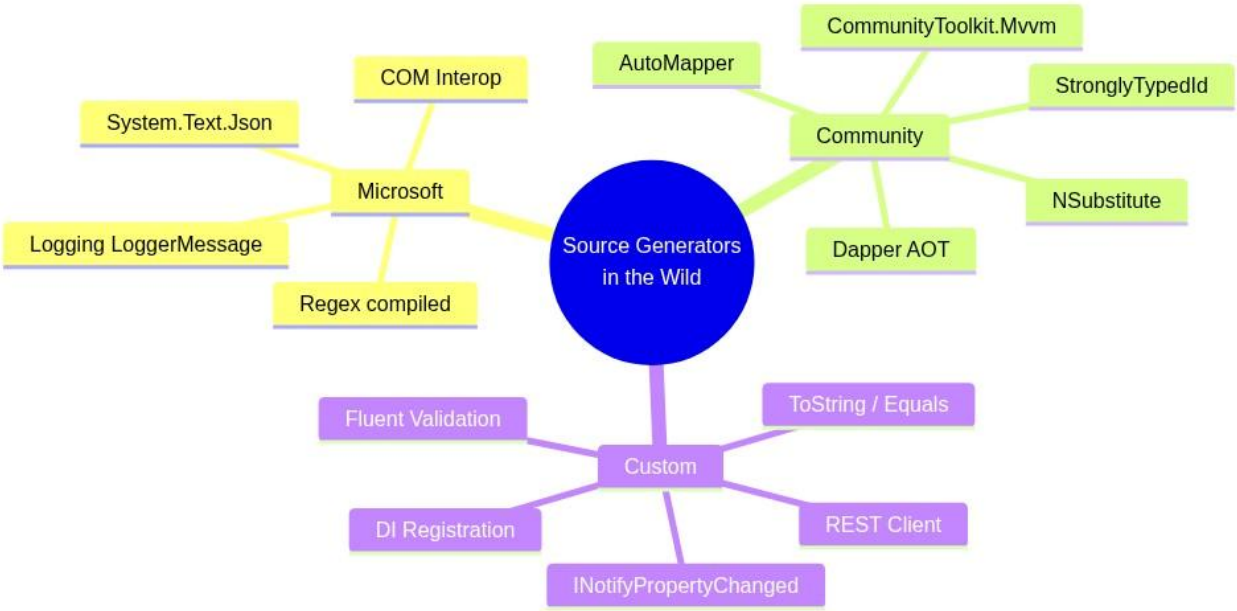
Common Gotchas (1)

Issue	Problem	Solution
Non-partial class	Generator can't add members	Require partial keyword; emit diagnostic
Namespace collisions	Generated types clash	Prefix with global:: in generated code
Nullable context	Generated code ignores NRTs	Add #nullable enable at top of generated file
Caching failures	Data types not equatable	Implement IEquatable<T> on model types
Build errors in generated code	Wrong emitted syntax	Use Roslyn's SyntaxFactory for complex output

Common Gotchas (2)

Issue	Problem	Solution
Slow IDE	Non-incremental generator	Migrate to IIncrementalGenerator
Legacy VS versions	Out-of-process not supported	Use netstandard2.0 for VS 2022 < 17.7

Real-World Generator Ecosystem



Summary

Source generators are the right tool when:

- You have repetitive code patterns that follow a template
- You need AOT compatibility (no reflection at runtime)
- You want compile-time verification of your patterns
- You need to integrate with attributes or naming conventions

Always use IIncrementalGenerator — it's the modern standard.

```
// The pattern with attributes in three lines:
var provider = context.SyntaxProvider.ForAttributeWithMetadataName(...)
                        .Where(x => x is not null);
context.RegisterSourceOutput(provider, GenerateCode);
```

Resources

- 📖 Incremental Generators Cookbook — Microsoft
- 🔬 Roslyn SDK Samples
- 🧪 Source Generator Testing Guide
- 🎯 CommunityToolkit MVVM Generator (reference impl)
- 📋 Samples (this repo)

Q & A

Common questions:

- *"When should I NOT use a source generator?"* — When runtime flexibility is needed, or when a T4 template or Roslyn analyzer suffices.
- *"Can generators reference NuGet packages?"* — Yes, but be careful with transitive dependencies and package compatibility.
- *"Why did generators require netstandard2.0?"* — Legacy VS versions ran generators in-process. **VS 2022 17.7+ / VS 2026+ run out-of-process** — you can now target modern .NET! ✅
- *"Can generated code call back into my code?"* — Yes, generated partial classes are compiled together with your code.
- *"How do I ship a generator?"* — Pack it as a NuGet package with the analyzers/dotnet/cs folder structure.

Thank You!

Developing and Debugging Source Generators

Enjoy *BASTA! Spring 2026*!!

Slides and samples: **github.com/CNILearn/bastaspring2026**

```
[Generator]
public class YourNextGenerator : IIncrementalGenerator
{
  // The boilerplate writes itself. 🚀
}
```

Wir bitten um
dein Feedback!

BITTE BEWERTEN SIE UNS!

entwickler.de